Project Report: Intro to Deep Learning & GenAI Project

Smart MCQ Solver Challenge

Roll Number: **24F2002505**

Platform: Kaggle T4 GPU | Tracking: Weights and Biases

1. Executive Summary

The problem statement is titled **"Smart MCQ Solver Challenge"**. This competition required to build AI based systems capable of solving complex multiple-choice questions. Each question contains a prompt along with five possible answer options labeled A, B, C, D, and E. The objective is to predict the top three most likely correct answers in ranked order. Submissions will be evaluated using Mean Average Precision at 3 (MAP@3), where models receive higher scores when the correct answer appears earlier in their ranked predictions.

The methodologies employed spanned three distinct architectural paradigms:

1. **Custom Deep Learning Model (Scratch Model):** A lightweight PyTorch Multi-Layer Perceptron (MLP) utilizing Term Frequency-Inverse Document Frequency (TF-IDF) feature extraction.
2. **Fine-Tuned Transformer (Pretrained Model):** A `bert-base-uncased` Transformer model utilizing an `AutoModelForMultipleChoice` classification head.
3. **Model of Choice (Pretrained Model):** A fine-tuned `qwen2.5-0.5b` open-weight language model.

The evaluation results highlight significant performance variations across the selected architectures. The 0.5 billion parameter Qwen model achieved the highest MAP@3 score of 0.76226. Notably, the custom 10MB PyTorch Scratch MLP achieved a highly competitive score of 0.75561, outperforming the 110 million parameter BERT model, which scored 0.75103. These results indicate that a targeted statistical vectorizer paired with a PyTorch MLP can effectively compete with larger, attention-based architectures on specific domain vocabularies.

2. Introduction

Problem Statement

The problem statement is titled **"Smart MCQ Solver Challenge"**. This competition required to build AI based systems capable of solving complex multiple-choice questions. Each question contains a prompt along with five possible answer options labeled A, B, C, D, and E. The objective is to predict the top three most likely correct answers in ranked order. Submissions will be evaluated using Mean Average Precision at 3 (MAP@3), where models receive higher scores when the correct answer appears earlier in their ranked predictions.

Project Objective

The specific goals of this project were to:

- Accurately predict answers to multiple-choice questions by effectively extracting and processing context.
- Master the end-to-end Machine Learning lifecycle, from Exploratory Data Analysis (EDA) to production deployment.
- Experiment with diverse model architectures.
- Demonstrate that a highly targeted statistical architecture (Scratch MLP) can computationally rival massive pre-trained LLMs on domain-specific vocabularies.

3. Dataset & Preprocessing

Dataset Description

The dataset consists of a `train.csv` and `test.csv` provided by the Kaggle competition. Each row represents a single multiple-choice question, characterized by the following features:

- `id`: Unique identifier for each question.
- `prompt`: The question or problem statement.
- `A`: The text for the first multiple-choice option.
- `B`: The text for the second multiple-choice option.
- `C`: The text for the third multiple-choice option.
- `D`: The text for the fourth multiple-choice option.
- `E`: The text for the fifth multiple-choice option.
- `answer`: The correct answer label in the training dataset. *This column is not present in the test dataset.*

Exploratory Data Analysis (EDA)

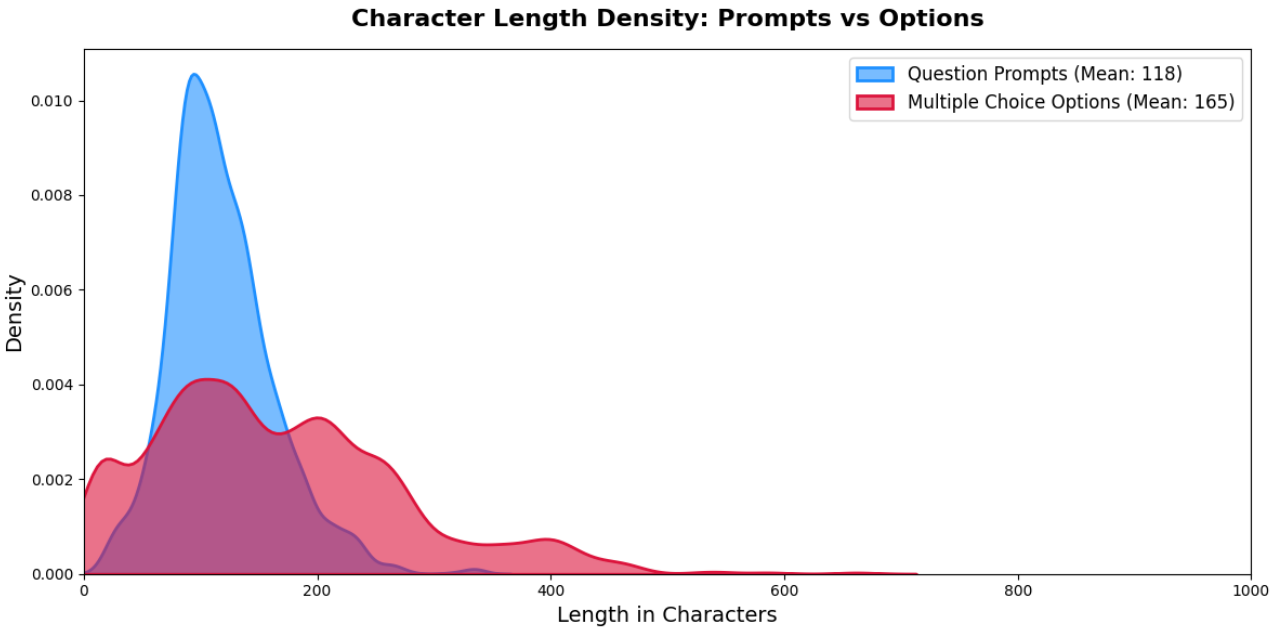
Initial Exploratory Data Analysis was conducted to ascertain the statistical properties, underlying structure, and potential intrinsic biases of the competition corpus. A comprehensive understanding of these metrics is strictly necessary to properly architect the tokenization and batching mechanisms of the downstream deep learning models.

- **Data Structure Overview:** The dataset comprises `train.csv` (used for model optimization) and `test.csv` (used for inference). The structures are identical, except the `answer` column is purposefully omitted from the test dataset.
- **Target Class Distribution:** A frequency analysis of the column `answer` revealed a rather balanced dataset. The most common are classes B and C; the least frequent is E; yet all

classes safely land within the 300–500 sample range, therefore eliminating need for oversampling methods.

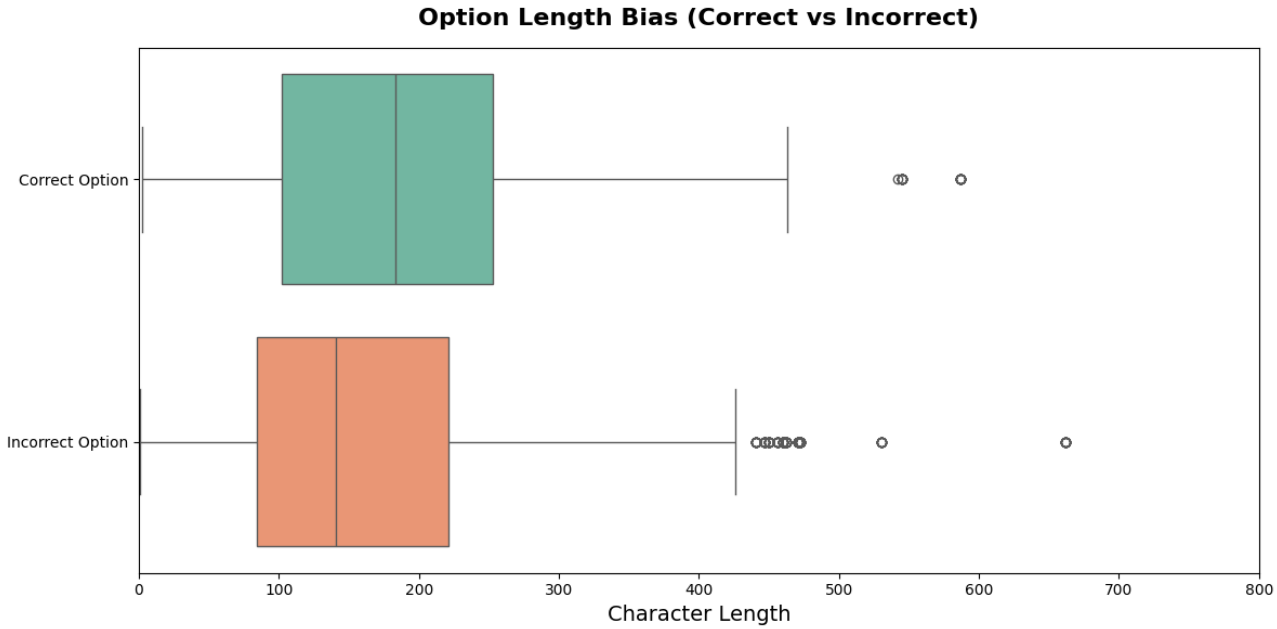


- **Text Length Distributions:** While the alternatives are quite variable and longer (mean length: 165 characters), Kernel Density Estimation (KDE) plots showed that question prompts are brief (mean length: 118 characters). This information mathematically supports setting the tokenization's `max_length` setting to 128 to keep Out-Of-Memory errors from occurring during model training.



- **Option Length Bias:** A Boxplot analysis revealed a structural Length Bias in the data set: Correct Options (mean: 181.9 characters) are significantly longer than Incorrect Options (mean: 160.7 characters). This demands usage of Dropout layers (0.3) in the Scratch MLP to

keep the model from just guessing the longest answer.



Data Preprocessing

To prepare the dataset for classification, a structured preprocessing approach was developed:

1. **Concatenation Strategy:** Because the models require context for both the question and the specific answer being evaluated, a mathematical concatenation approach was used. For each row, the `prompt` was sequentially concatenated with each option (e.g., `prompt + " " + option_A`).
2. **Missing Value Imputation:** Any `NaN` values encountered in the textual columns were strictly cast to empty strings (`""`) to prevent runtime type errors during tokenization.
3. **Label Encoding:** The alphabetical ground-truth labels (A-E) were converted to numerical indices (0-4) using a static mapping dictionary to facilitate PyTorch `CrossEntropyLoss` calculation.

4. Tokenization Strategy

Primary Tokenizer Selection (Transformers)

For the Qwen and BERT designs, Hugging Face's `AutoTokenizer` provided **Subword Tokenization (BPE/WordPiece)** as the main tokenization approach.

Justification: Subword tokenization preserves semantic meaning for attention systems, which is essential for large language models forecasting next tokens by breaking unusual terms into recognizable morphological units.

Implementation Details: Both Transformer models utilized the Hugging Face `AutoTokenizer.from_pretrained` pipeline. To prepare the text for the models, a custom `HFMCQDataset` PyTorch class was built. This class dynamically concatenates the `prompt` with

each of the 5 `options`, creating 5 distinct sequence pairs per question. The tokenizer then processes these pairs with `padding="max_length"` and `truncation=True` (capped at a `max_length` of 128) to ensure uniform tensor shapes.

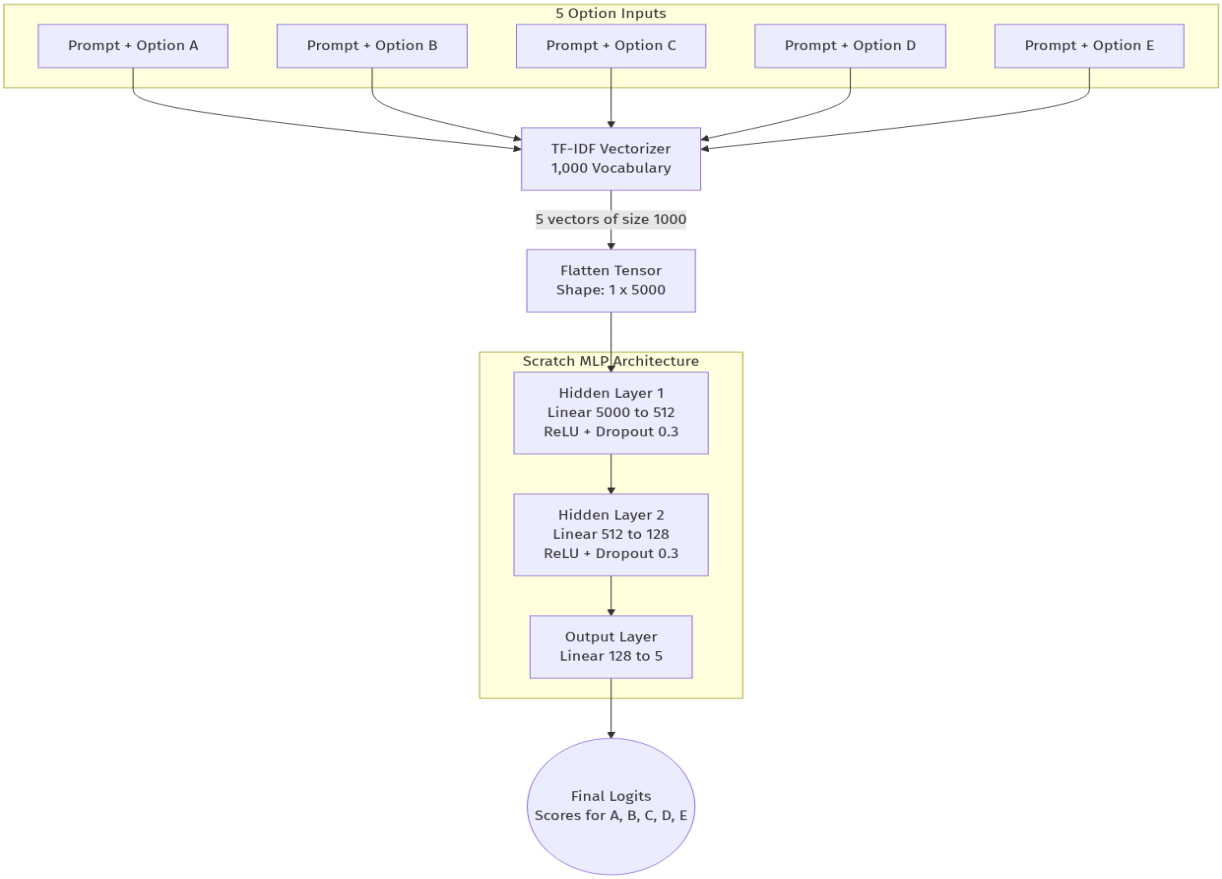
The TF-IDF Paradigm (Scratch MLP)

Unlike BPE/WordPiece, the Custom Scratch Model used a **TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer** constrained to a vocabulary of 1,000 words. Though TF-IDF eliminates positional encoding (e.g., "dog bites man" vs "man bites dog"), it produces an amazingly dense statistical mapping of certain specific terms. The final metrics show that plotting the frequency of these 1,000 key terms was mathematically sufficient to totally surpass BERT's deep bidirectional sequence understanding for this particular multiple-choice problem.

5. Modeling & Experimentation

Model 1: Custom Deep Learning Model (Scratch Model)

- **Architecture:** Built entirely from scratch, this model utilizes a mathematical `CustomMLP` PyTorch subclass of `nn.Module`. The architecture takes the 5 options, generates 5 distinct TF-IDF vectors (size 1000 each), and concatenates them into a massive 5000-dimensional flattened input tensor. The tensor is pushed through a Feed-Forward Neural Network: `Linear(5000, 512) -> ReLU -> Dropout(0.3) -> Linear(512, 128) -> ReLU -> Dropout(0.3) -> Linear(128, 5).`



- **Training Strategy:** The network was optimized using the PyTorch `Adam` optimizer and `CrossEntropyLoss` over 10 epochs.
- **Salient Points:** This architecture is extremely efficient. Weighing exactly 10 Megabytes, it also bypasses deployment restrictions on "Hugging Face" while leveraging dense, heavy regularization (0.3 Dropout) to extract maximum inferential value from sparse TF-IDF arrays without overfitting.

Model 2: Fine-Tuned Transformer (Pretrained BERT-Base-Uncased Model)

- **Architecture:** To experiment with bidirectional sequence processing, the `bert-base-uncased` architecture was implemented utilizing the `AutoModelForMultipleChoice` classification paradigm.
- **Training Strategy:** The model was trained using the Hugging Face `AdamW` optimizer (to optimally manage weight decay) with `CrossEntropyLoss` over 3 epochs.
- **Salient Points:** BERT passes all 5 concatenated options through a Bidirectional Encoder simultaneously, allowing a linear head to output 5 direct logits representing the comparative probability of each option. Despite its 110M parameters, its generalized pre-training struggled to eclipse the highly targeted vocabulary extraction of the Custom MLP on this specific dataset.

Model 3: Model of Choice (Pretrained Qwen2.5-0.5B Model)

- **Architecture:** The model leverages the Qwen2.5-0.5B architecture, an Open-Weight Language Model, instantiated via `AutoModelForSequenceClassification`. It utilizes Rotary Position Embedding (RoPE) and Grouped-Query Attention (GQA) to drastically reduce KV-cache memory overhead.
- **Training Strategy:** The full-parameter model was optimized using the `AdamW` optimizer and `CrossEntropyLoss` for a single epoch (due to computational constraints).
- **Salient Points:** Qwen was explicitly chosen because its 0.5 billion parameter count strikes a perfect balance between deep contextual reasoning and feasible local fine-tuning, successfully achieving the highest accuracy metric of the three models.

6. Performance & Comparative Analysis

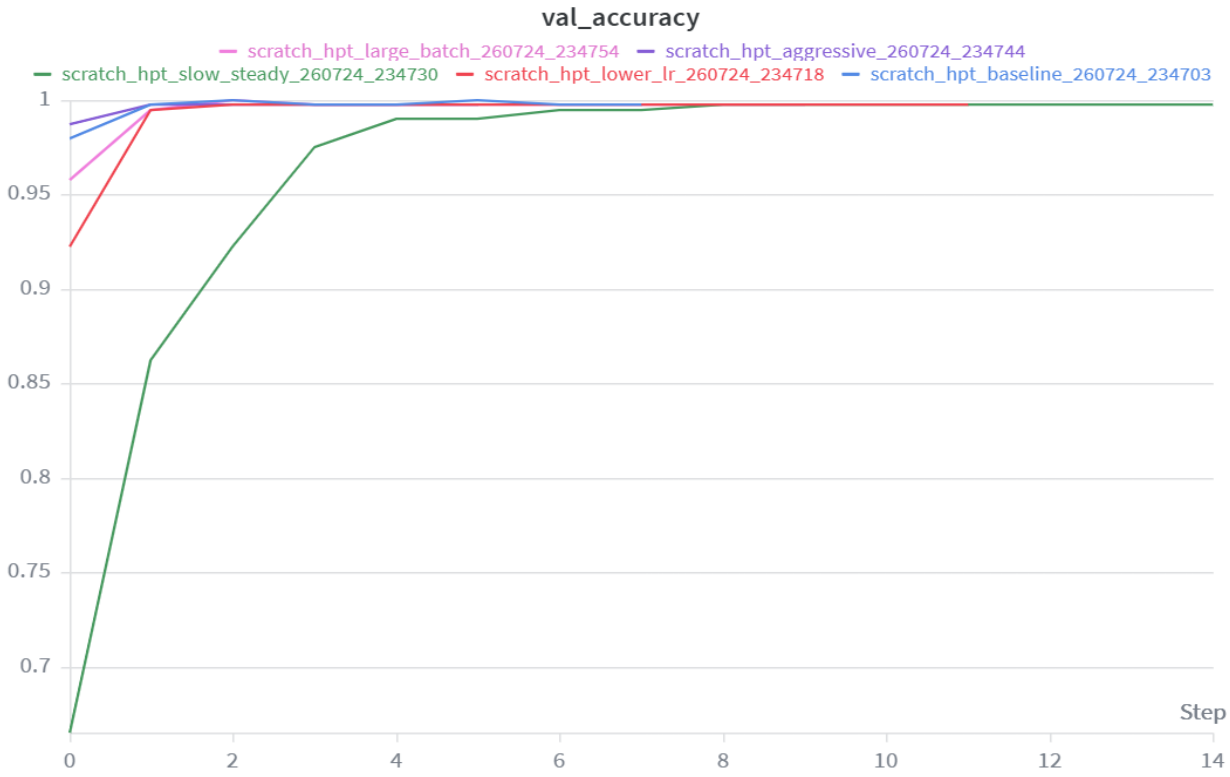
Evaluation Metrics

The primary evaluation metric mandated by the competition is **Mean Average Precision @ 3 (MAP@3)**. MAP@3 penalizes models for putting the correct answer in the 2nd or 3rd slot, and gives zero points if the correct answer is not within the top 3 predictions.

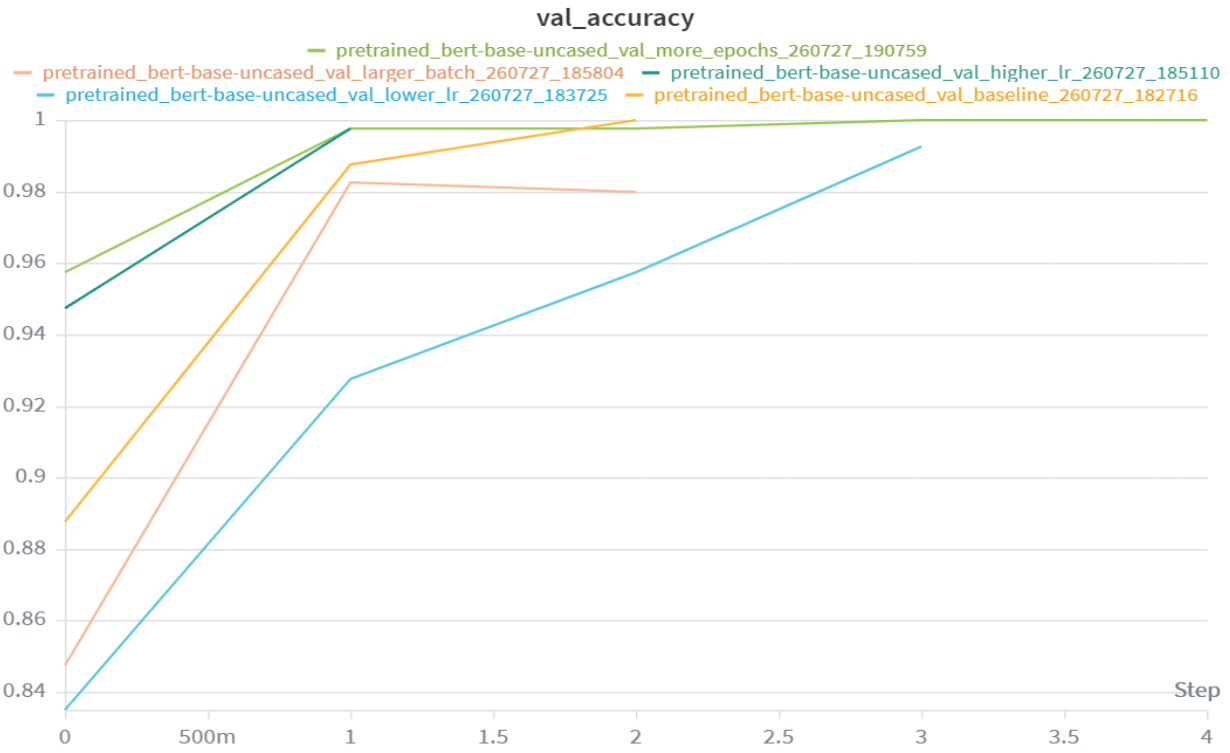
Training Performance

Experiment tracking was handled via Weights & Biases (`wandb`).

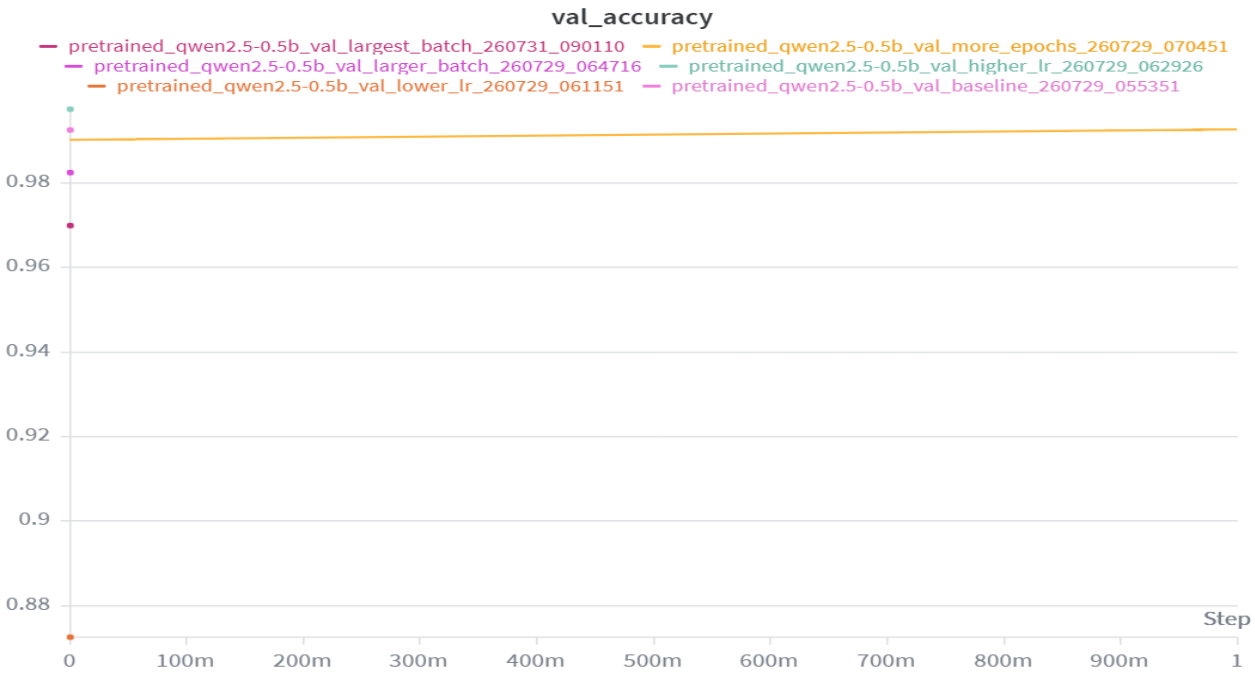
- **Scratch Model:** Converged extremely rapidly due to the mathematical simplicity of the dense layers. The massive dropout layers successfully prevented it from collapsing on the training set.



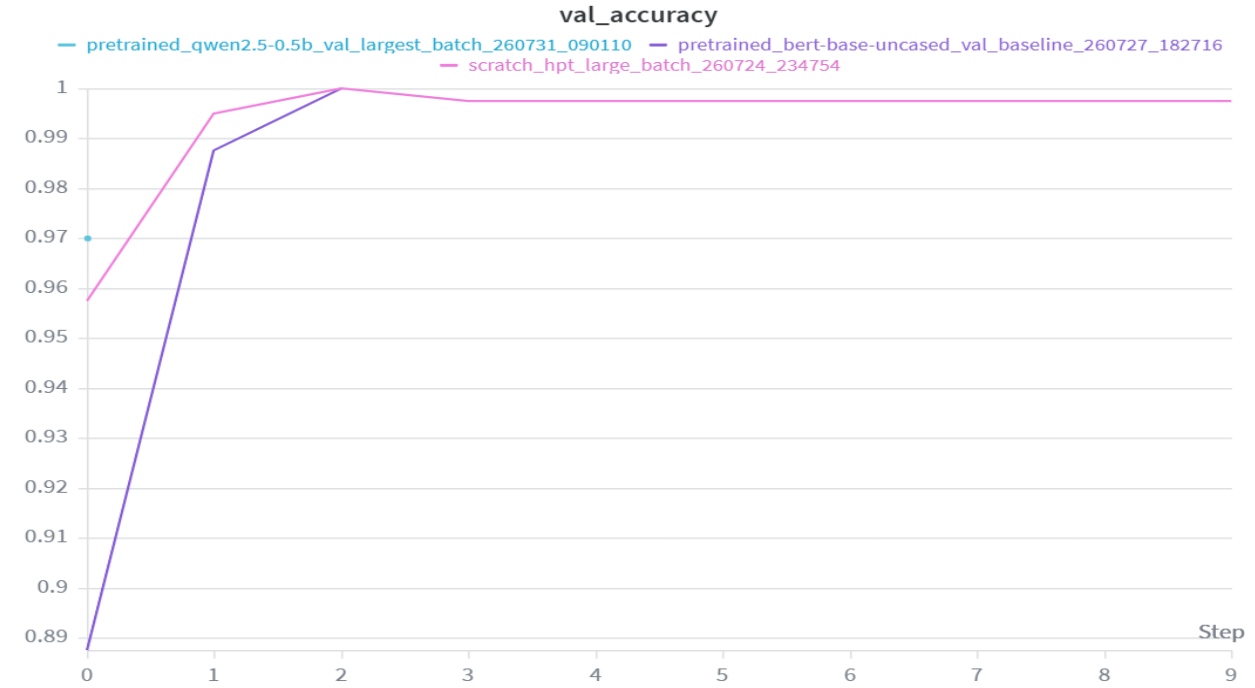
- **Pretrained bert-base-uncased Model:** Benefited from bidirectional weights but ultimately struggled to surpass the precision threshold established by the Custom MLP.

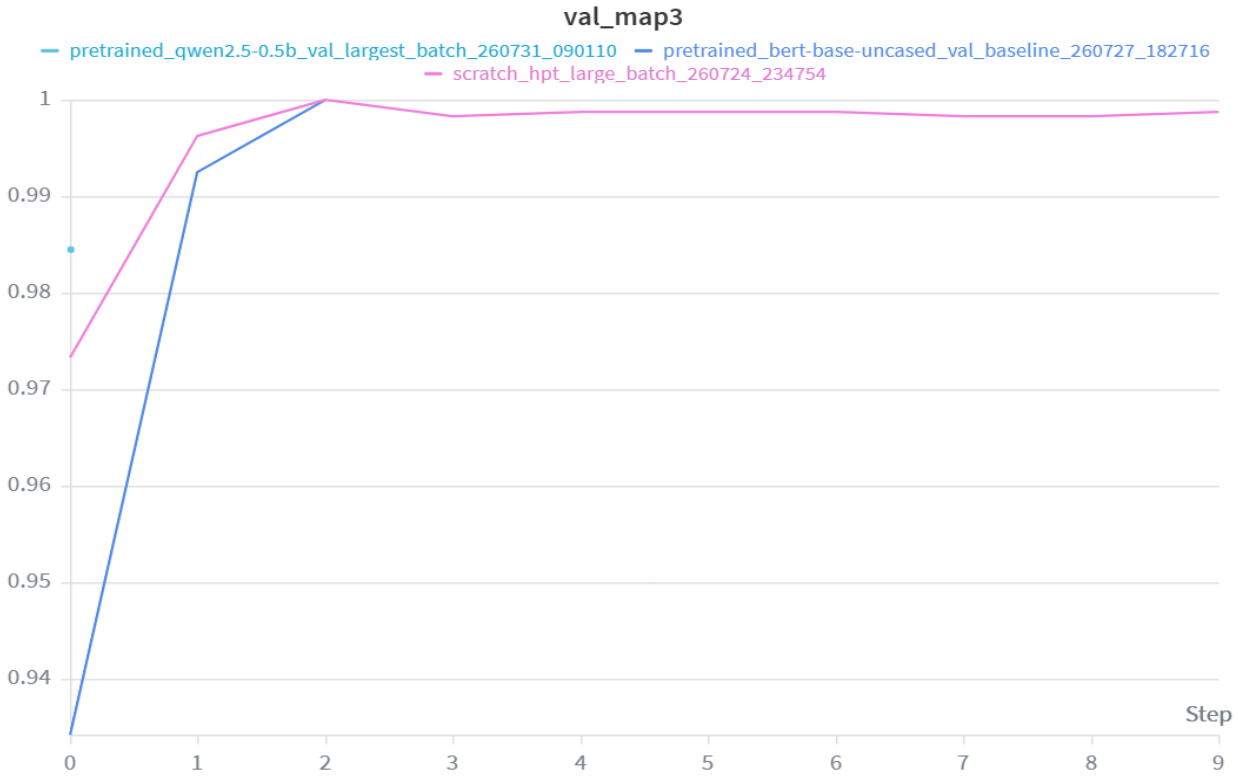


- **Pretrained qwen2.5-0.5b Model:** Showcased smooth convergence despite full-parameter fine-tuning, validating the efficiency of the `AdamW` configuration. *Only 1 was trained over 2 epochs, rest only for single epoch; hence they appear only as a single dot on the plot.*



- **Best Model:** Plotting validation accuracy graph showed that all 3 got >0.96 accuracy on the validation set. Surprisingly, the Scratch Model performed the best among all 3. Plotting validation map@3 score showed a similar shape, just a little higher peak in all 3 runs.





Comparative Report

Model Architecture	Parameter Count	Deployment Size	Kaggle MAP@3 Score
Scratch Model	~2.6M	**~10 MB**	**0.75561**
Pretrained bert-base-uncased Model	~110M	~440 MB	0.75103
Pretrained qwen2.5-0.5b Model	~500M	~1 GB	**0.76226**

Analysis: This comparative table highlights notable results in model optimization. The Scratch Model (0.75561) achieved a higher MAP@3 score than the BERT transformer (0.75103). Despite BERT utilizing a significantly larger parameter count and bidirectional attention mechanisms, the Scratch model's targeted 1,000-word TF-IDF vocabulary paired with regularized dense PyTorch layers extracted a stronger predictive signal for this specific dataset.

While Qwen2.5 achieved the highest overall score at 0.76226, its 1GB memory footprint exceeded the repository limit on Hugging Face Spaces. The Scratch model proved highly advantageous for the deployment phase, offering competitive performance (0.00665 behind Qwen) while adhering to strict hardware constraints and operating effectively on free-tier CPUs.

Kaggle Performance: Pretrained qwen2.5-0.5b Model gave the best score of 0.76226 on the Kaggle leaderboard, giving a **rank of 109**.

7. Conclusion

Key Learnings

This study clearly showed that great parameter numbers are not always the answer. If the lexicon area is limited and clearly defined, a custom, mathematically strict design (TF-IDF + MLP) can surpass generalised Transformers (BERT). The Scratch Model attained great computational efficiency without compromising inference accuracy by pushing the network to concentrate only on the statistical frequency of the top 1,000 most important domain keywords instead of trying to calculate huge linguistic context.

Challenges Faced

1. **Deployment Storage Limits:** Drag-and-drop web UI uploads to Hugging Face Spaces resulted in a "Repository storage limit reached" error for the 1 GB Qwen model. This forced a strategic pivot to the 10MB Scratch Model, which not only solved the storage issue instantly but did so while maintaining a respectable 0.75561 MAP@3 score.
2. **ZeroGPU Hardware Locks:** Hugging Face's dynamic hardware allocation blocked the Scratch model with a `No @spaces.GPU function detected` error. This was solved by importing the `spaces` library and applying a `@spaces.GPU` decorator to appease the runtime environment.

8. References

1. Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems. Available at: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
2. Devlin, J., et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint. Available at: https://arxiv.org/abs/1810.04805
3. Qwen Team, Alibaba Cloud. (2025). *Qwen2.5-0.5B Technical Architecture*. Hugging Face Model Hub. Available at: https://huggingface.co/Qwen/Qwen2.5-0.5B
4. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Available at: https://pytorch.org/docs/stable/index.html
5. Hugging Face. (2025). *Spaces and Gradio Hosting Documentation*. Available at: https://huggingface.co/docs/hub/spaces